The Islamia University of Bahawalpur

Faculty of Computer Science

Department of Information Technology

Software Design Document

Face Recognition-Based Classroom Attendance System

Submitted By: Muhammad Asghar

Roll No: S23BINFT1E02019

Semester: 7th (1E)

Submitted To: Pro.Dr Dost Muhammad Khan

Department: Information Technology

Date: June, 2026

Summary

This Software Design Document (SDD) presents the comprehensive architectural blueprint for the Face Recognition-Based Classroom Attendance System — a standalone web-based application developed for The Islamia University of Bahawalpur, Department of Information Technology.

The system replaces conventional manual roll-call and paper-register methods with an intelligent, contactless attendance solution powered by computer vision and facial recognition technology. Upon a student entering the classroom, the system automatically detects and identifies their face through a live camera feed, logs the attendance record with a timestamp, and updates the central database — without requiring manual input from the teacher or student.

The platform provides a secure, role-based web dashboard for administrators and teachers to register students, manage class schedules, monitor real-time attendance, generate detailed reports, and configure system settings. Students access a self-service portal to view personal attendance history and receive low-attendance alerts.

The AI engine is built on Python, OpenCV, and the face_recognition library. The backend uses Flask with SQLite. This document defines the system architecture, component design, data model, UI specification, and coding conventions that guide the complete development lifecycle.

1. Application Architecture

1.1 Architectural Overview

The system follows a three-tier Client-Server Architecture with an embedded real-time face recognition subsystem:

- Tier 1 — Presentation Layer: A web-based responsive dashboard (Next.js / HTML5-CSS3) for administrators, teachers, and students. Communicates with the backend via RESTful HTTP APIs.
- Tier 2 — Application Layer: Flask backend handling all business logic — user authentication, face recognition orchestration, attendance session management, report generation, and API responses.
- Tier 3 — Data Layer: SQLite database providing persistent storage for student profiles, face encodings, class schedules, attendance logs, and user credentials.

The face recognition subsystem operates as a dedicated Python module within the Application Layer. It interfaces with the classroom camera via OpenCV, processes video frames in real time, encodes detected faces using the face_recognition library, and queries the Data Layer to match encodings against registered students. Recognized students are immediately passed to the attendance logging module.

1.2 Components of Architecture

1.2.1 Proposed Models

The following data models represent the core entities managed by the system:

- UserModel — Stores authentication and role data for Administrators and Teachers.
- StudentModel — Stores academic profile, face encoding metadata, and enrollment status.
- ClassModel — Stores class schedule, assigned teacher, subject, and time slots.
- AttendanceModel — Stores every attendance event with student ID, class ID, session ID, timestamp, status, and face confidence score.
- SessionModel — Tracks active attendance sessions initiated by teachers.

1.2.2 Proposed Views (Screens)

- LoginPage — Role-based authentication entry point for Admins and Teachers.
- AdminDashboard — Overview of system statistics, student counts, class schedules.
- StudentManagementPage — Add, view, update, and deactivate student profiles.
- FaceEnrollmentPage — Webcam-based face capture and encoding interface.
- ClassManagementPage — Create and manage class schedules.
- AttendanceSessionPage — Teacher-controlled live attendance session with real-time recognition feed.
- ReportsPage — Filterable attendance report table with CSV export.
- StudentPortalPage — Student-facing attendance history and low-attendance alerts.

1.2.3 Backend Modules (Flask)

Module	Responsibility
face_recognition_engine.py	Core AI module. Interfaces with OpenCV for camera frame capture, encodes detected faces using face_recognition library, and matches against stored 128-dim encodings. Returns matched student ID and confidence score.
auth.py	User authentication, session management via Flask-Login, and password hashing with Werkzeug PBKDF2.
student_manager.py	CRUD operations for student profiles in the Students table. Manages face enrollment status flag.
class_manager.py	Handles class schedule creation, assignment, conflict detection, and deactivation.
attendance_manager.py	Records attendance events from the recognition engine. Prevents duplicate entries per session. Marks absent students on session close.
session_controller.py	Starts and ends attendance sessions. Links all attendance records to the active SessionID.
report_generator.py	Queries Attendance and Classes tables to compute statistics (present count, absent count, percentage). Produces CSV output.
notification_engine.py	Checks student attendance percentages against the configured threshold and generates low-attendance alert records.
routes.py	Defines all RESTful API endpoint routes and maps them to controller functions using Flask Blueprints.

1.3 API Endpoints

Method	Endpoint	Description
POST	/api/auth/login	Validates credentials and returns session token with role.
POST	/api/auth/logout	Terminates the authenticated session.
GET	/api/students	Returns list of all registered students (Admin only).
POST	/api/students	Creates a new student profile.
PUT	/api/students/{id}	Updates an existing student record.
POST	/api/students/{id}/enroll	Triggers face capture and encoding for a student.
GET	/api/classes	Returns all class schedules accessible to the authenticated user.
POST	/api/classes	Creates a new class schedule (Admin only).
POST	/api/sessions/start	Starts an attendance session for a class (Teacher only).
POST	/api/sessions/end	Ends the active session; marks absent students.
GET	/api/attendance/report	Returns filtered attendance report with statistics.
GET	/api/attendance/export	Downloads attendance report as CSV.
GET	/api/student/portal	Returns personal attendance data for authenticated student.

1.4 Coding Conventions

Python (Backend & AI Engine)

- All functions and variables use snake_case (e.g., encode_face, class_id, student_roll).
- Classes use PascalCase (e.g., FaceRecognitionEngine, AttendanceManager).
- All database operations use parameterized queries to prevent SQL injection.
- Flask Blueprints used to separate auth, student, class, attendance, and report route modules.
- Face encoding operations run in a background thread to prevent blocking the web server.

Frontend (JavaScript / Next.js)

- Variables and functions use camelCase (e.g., startSession, studentId, attendancePercent).
- Components use PascalCase (e.g., AttendanceSessionPage, StudentPortal).
- WebSocket or polling (every 2 seconds) used for real-time attendance feed updates during live sessions.
- Role-based route guards prevent unauthorized access to admin and teacher pages.

2. Data Model Schema

2.1 Conceptual Schema

The system uses an SQLite relational database with five core tables: Users, Students, Classes, Sessions, and Attendance. Referential integrity is enforced through foreign key constraints. All tables include timestamp fields for full auditability. Soft-delete policies (IsActive flags) preserve historical data when records are deactivated.

2.2 Entity Details

2.2.1 Users Table

Stores authentication and role information for all system users (Administrators and Teachers).

Column Name	Data Type	Constraints	Description
UserID	INTEGER	PK, AUTO	Unique system identifier
Name	TEXT	NOT NULL	Full name of the user
Email	TEXT	NOT NULL, UNIQUE	Login email address
PasswordHash	TEXT	NOT NULL	PBKDF2/bcrypt hashed password
Role	TEXT	NOT NULL	Admin or Teacher
IsActive	BOOLEAN	DEFAULT TRUE	Account active status
CreatedAt	DATETIME	DEFAULT NOW	Account creation timestamp

2.2.2 Students Table

Stores academic and biometric enrollment information for all registered students.

Column Name	Data Type	Constraints	Description
StudentID	INTEGER	PK, AUTO	Unique student identifier
Name	TEXT	NOT NULL	Student full name
RollNumber	TEXT	NOT NULL, UNIQUE	University roll number
Department	TEXT	NOT NULL	Student department
Semester	TEXT	NOT NULL	Current semester
Session	TEXT	NOT NULL	Academic session
FaceImagePath	TEXT	NULLABLE	Path to enrollment image files
FaceEncoding	BLOB	NULLABLE	128-dimensional face encoding vector
IsEnrolled	BOOLEAN	DEFAULT FALSE	Face enrollment completion status
IsActive	BOOLEAN	DEFAULT TRUE	Student active/deactivated status

2.2.3 Classes Table

Stores class schedule information including assigned teacher, subject, and time slots.

Column Name	Data Type	Constraints	Description
ClassID	INTEGER	PK, AUTO	Unique class identifier
ClassName	TEXT	NOT NULL	Name of the class
Subject	TEXT	NOT NULL	Subject name
TeacherID	INTEGER	FK → Users.UserID	Assigned teacher reference
Classroom	TEXT	NOT NULL	Room or lab number
Schedule	TEXT	NOT NULL	Day(s) and time slot
Semester	TEXT	NOT NULL	Academic semester
IsActive	BOOLEAN	DEFAULT TRUE	Class active status

2.2.4 Sessions Table

Tracks attendance session lifecycle for each class.

Column Name	Data Type	Constraints	Description
SessionID	INTEGER	PK, AUTO	Unique session identifier
ClassID	INTEGER	FK → Classes.ClassID	Referenced class
StartedAt	DATETIME	NOT NULL	Session start timestamp
EndedAt	DATETIME	NULLABLE	Session end timestamp (NULL if active)
Status	TEXT	NOT NULL	Active or Closed
InitiatedBy	INTEGER	FK → Users.UserID	Teacher who started the session

2.2.5 Attendance Table

The primary transactional table storing every attendance event logged by the system.

Column Name	Data Type	Constraints	Description
AttendanceID	INTEGER	PK, AUTO	Unique attendance record identifier
StudentID	INTEGER	FK → Students.StudentID	Referenced student
ClassID	INTEGER	FK → Classes.ClassID	Referenced class
SessionID	INTEGER	FK → Sessions.SessionID	Referenced session
Date	DATE	NOT NULL	Date of attendance
Time	TIME	NOT NULL	Time of face recognition event
Status	TEXT	NOT NULL	Present or Absent
Confidence	REAL	NULLABLE	Face match confidence score (0.0–1.0)

2.3 Entity Relationship Overview

Users are assigned to Classes via the TeacherID foreign key. Classes generate Sessions, and Sessions contain Attendance records. Each Attendance record references a Student and a Class. The Students table holds face encodings used by the recognition engine at runtime. Referential integrity constraints prevent orphaned attendance records when classes or students are deactivated (soft-delete via Is Active flags preserves all historical data).

3. User Interface

3.1 Design Philosophy

The interface follows a professional, institutional design language suitable for a university environment. It prioritises clarity, role-based access separation, and minimal learning curve for non-technical users (teachers and administrators).

- Framework: Next.js with HTML5/CSS3 fallback for maximum browser compatibility.
- Responsiveness: Fully responsive layout supporting desktop, tablet, and mobile browsers.
- Primary colour: Forest Green — communicates institutional trust and academic professionalism.
- Accent colour: Amber — used for alert indicators and low-attendance warnings.
- Typography: 'Inter' or 'Roboto' for clean readability in data-dense tables and dashboards.
- Role-based navigation ensures admins, teachers, and students each see only their relevant sections.

3.2 Screen Details and Controls

Screen (UC)	Data Displayed	UI Controls	Input Constraints & Logic
Login (UC001)	Login form	Email TextField, Password field, Login button, Forgot Password link	Fields cannot be empty. Failed attempts display error. Account deactivation message shown if applicable.
Face Enrollment (UC002)	Live webcam feed, enrollment progress	Webcam preview, Capture button (auto 5 frames), Re-capture option, Confirm button	Minimum 5 frames required. Quality check for partial face / low light. Overwrite confirmation if re-enrolling.
Attendance Session (UC003)	Live camera feed, real-time recognized student list with name and roll number	Start Session button, live recognition feed, End Session button, Manual Override option	Session only starts for teacher's assigned class. Camera activation tied to session state. Duplicate entries suppressed.
Report Generation (UC004)	Filterable attendance table: student, sessions, present/absent counts, percentage	Class dropdown, Subject dropdown, Date Range pickers, Roll Number field (optional), Export CSV button	At least one filter required. Empty result displays informational message. Export generates CSV download.
Student Portal (UC005)	Per-subject attendance summary: sessions, present, absent, percentage. Low-attendance alert if below threshold	Subject filter, Date range filter, session detail drill-down	Read-only access. Alert banner displayed if any subject below 75%. Login required.
Class Management (UC006)	Class schedule table: name, subject, teacher, room, day/time	Create Class form, Edit button per row, Deactivate button, conflict validation feedback	Teacher must exist in system. Conflict detection on same teacher/time slot. Required fields enforced.

3.3 Architectural Diagrams

The following diagrams are to be inserted as part of the final design deliverable:

- Figure 1: Three-Tier System Architecture Diagram — showing Presentation, Application, and Data layers with the face recognition subsystem highlighted within the Application Layer.
- Figure 2: Face Recognition Pipeline Flow — from camera frame capture through OpenCV preprocessing, face encoding, database matching, and attendance logging.
- Figure 3: Entity Relationship Diagram — depicting the five SQLite tables (Users, Students, Classes, Sessions, Attendance) with foreign key relationships.
- Figure 4: Sequence Diagram — UC003 Automated Attendance Marking, showing the interaction between Teacher, Flask backend, face recognition engine, database, and frontend dashboard.
- Figure 5: Class Diagram — showing the major backend modules (FaceRecognitionEngine, AttendanceManager, SessionController, ReportGenerator) and their method signatures.

4. Non-Functional Design Considerations

4.1 Performance

The face recognition engine processes and identifies a student's face within 2–3 seconds per frame under standard indoor lighting. The web dashboard loads and responds within 2 seconds under normal network conditions. The system processes a classroom of up to 60 students within a single attendance session without significant performance degradation. Face encoding operations run in a background thread to avoid blocking the Flask web server.

4.2 Recognition Accuracy

The face recognition model achieves a minimum identification accuracy of 95% for registered students under normal classroom lighting. A minimum of 5 face images per student are captured during enrollment. A configurable confidence threshold rejects matches below the minimum score to prevent false positives. Unmatched detections are flagged for manual review.

4.3 Security

All user passwords are stored using bcrypt or Werkzeug PBKDF2 one-way hashing. Role-based access control (RBAC) is implemented: teachers cannot access other teachers' class data; students can only view their own records. All API endpoints require valid authenticated session tokens. Input validation and parameterized queries prevent SQL injection. Biometric face data is stored securely and not shared with any third party.

4.4 Reliability and Availability

The system maintains 99% operational availability during scheduled university hours (6:00 AM – 10:00 PM). Attendance records already logged are not affected by temporary camera interruptions. Database transaction management ensures atomicity of attendance records, preventing partial or duplicate entries. Regular database backups are performed.

4.5 Scalability

The system architecture supports the addition of new students, classes, departments, and academic sessions without structural database changes. The face recognition model supports retraining with new student data without affecting existing enrolled students. Flask Blueprints structure the backend to allow new modules to be added independently.

4.6 Maintainability

A modular software architecture with clear separation between the face recognition engine, business logic layer, and presentation layer ensures independent replaceability of components. All source code is documented with inline comments. The project is version-controlled on GitHub. An API reference document is maintained alongside the codebase.

4.7 Ethical and Legal Compliance

The collection and storage of biometric face data complies with applicable data privacy norms. Students are formally informed of biometric data collection during the enrollment process. Face image data is stored securely on the institutional server and is not shared with any external party. Data retention policies are aligned with institutional guidelines.

5. References

[1] IEEE Std 830-1998, Software Requirements Specifications, IEEE Computer Society, 1998.

[2] OpenCV Documentation, OpenCV Team, 2025. Available: https://docs.opencv.org

[3] face_recognition Library — A. Geitgey, GitHub, 2025. Available: https://github.com/ageitgey/face_recognition

[4] dlib C++ Library — D. King, dlib.net, 2025. Available: http://dlib.net

[5] Flask Documentation, Pallets Projects, 2025. Available: https://flask.palletsprojects.com

[6] SQLite Documentation, SQLite Consortium, 2025. Available: https://www.sqlite.org/docs.html

[7] Python 3.x Documentation, Python Software Foundation, 2025. Available: https://docs.python.org

[8] Next.js Documentation, Vercel Inc., 2025. Available: https://nextjs.org/docs

[9] OWASP Top Ten Web Application Security Risks, OWASP Foundation, 2024. Available: https://owasp.org

[10] Werkzeug Security Utilities, Pallets Projects, 2025. Available: https://werkzeug.palletsprojects.com

[11] I. Masi, Y. Wu, T. Hassner, and P. Natarajan, 'Deep Face Recognition: A Survey,' arXiv:1804.06655, 2018.

[12] G. B. Huang et al., 'Labeled Faces in the Wild,' University of Massachusetts, Technical Report 07-49, 2007.